

Advanced Architectural Framework for Image-Guided Robotic Surgery: A Direct Bridge Integrating 3D Slicer with PyBullet and MuJoCo Physics Engines

Introduction to Image-Guided Robotic Interventions

The paradigm of image-guided robotic interventions represents a highly transformative frontier in modern surgery, seamlessly blending advanced medical imaging with precision robotics to enhance surgical outcomes, refine tool-tip precision, and mitigate the impact of human physiological tremors¹. The evolution of these systems relies heavily on the integration of complementary situational awareness, a concept that demands real-time contextual models of patient anatomy and intra-operative devices¹. To achieve this, researchers increasingly depend on digital twins and dynamic physics simulations that allow for the interaction of robotic models within a virtual surgical environment prior to or during physical execution.

Historically, the development of intelligent image-guided robotic systems has faced a significant computational and architectural bottleneck: the lack of a unified, open-source framework capable of handling both high-fidelity medical image processing and complex multibody physics simulation³. The 3D Slicer platform stands as the de facto standard for medical imaging research, offering robust functionalities for segmentation, volumetric rendering, spatial registration, and surgical navigation¹. Simultaneously, physics engines like PyBullet, MuJoCo, and the Simulation Open Framework Architecture (SOFA) govern the domain of robot kinematics, dynamics, and soft-body interactions⁵. Previous integration attempts have predominantly relied on middleware frameworks such as the Robot Operating System (ROS) or OpenIGTLink to bridge these disparate domains¹. While functional for low-frequency data transfer, these middleware-heavy architectures introduce unacceptable latency, complexity, and data synchronization challenges for high-fidelity surgical simulation. This report provides an exhaustive analysis of the architectural designs, mathematical formulations, and software engineering required to construct a direct, low-latency bridge between 3D Slicer and advanced physics engines, fulfilling the specific objective of initiating with PyBullet and subsequently scaling to MuJoCo. Furthermore, it details the progression toward a full-fledged surgical simulator equipped with soft-body tissue deformations, haptic feedback mechanisms, and surgical control constraints, culminating in a comprehensive implementation roadmap for all programmatic objectives.

Critical Evaluation of Existing 3D Slicer and ROS2 Bridges

To design an optimal, direct bridge between 3D Slicer and a physics engine, it is imperative to

dissect the mechanical and architectural limitations of existing integration paradigms, particularly those leveraging ROS2 and OpenIGTLink. Implementations such as the ROS-IGTL bridge, SlicerROS2, and the AMBF (Asynchronous Multi-Body Framework) integration demonstrate the baseline feasibility of combining 3D Slicer's Medical Reality Modelling Language (MRML) scene graph with dynamic simulations¹.

The SlicerROS2 Middleware Paradigm

The SlicerROS2 module represents the current state-of-the-art in bridging imaging and robotics, encapsulating ROS2 communication mechanisms (topics, parameters, and the tf2 transformation tree) within custom MRML node structures³. By treating 3D Slicer as a native ROS2 node via the rclcpp C++ API, SlicerROS2 allows bidirectional data transfer between the robotic environment and the medical imaging platform³.

The architecture relies heavily on mapping ROS2 components to 3D Slicer using derived nodes such as vtkMRMLROS2NodeNode (for managing the execution model), vtkMRMLROS2PublisherNode, and vtkMRMLROS2Tf2LookupNode³. The tf2 library tracks coordinate frames in the robotic world, which SlicerROS2 queries and translates into vtkMRMLLinearTransformNode entities within Slicer's hierarchical transformation tree². Because the MRML scene is fundamentally not thread-safe, SlicerROS2 utilizes a Qt timer executing at 50 Hz (or up to 200 Hz for high-frequency updates) to "spin" the ROS node on Slicer's main thread and trigger callbacks¹.

Performance evaluations of the SlicerROS2 architecture indicate a highly stable round-trip delay (RTD) of approximately 11.2 to 20 milliseconds¹. While this delay falls well within the human-computer interaction threshold required to prevent visual perceptual lag, it exposes a critical vulnerability when applied to closed-loop haptic control and surgical simulation.

The Necessity of Bypassing Middleware

While the ROS2 bridge facilitates modularity and wide robot compatibility, introducing a publisher/subscriber middleware inherently caps the throughput and increases the architectural footprint. For high-fidelity surgical simulation—particularly those requiring bilateral teleoperation and kinesthetic haptic feedback—the control loop must operate at a minimum of 1000 Hz to ensure the rendering of stiff contacts without initiating unstable harmonic oscillations¹⁰. The serialization and deserialization overhead of ROS2 messages, coupled with the network stack latency, can induce micro-stutters and phase delays that compromise stability in admittance control schemes¹¹. Thus, circumventing ROS2 in favor of a direct memory-sharing or Python-bound Application Programming Interface (API) bridge between 3D Slicer and the selected physics engines represents a mandatory optimization for the proposed surgical simulator¹².

Objective 1: Architecting a Direct Slicer-Physics Bridge

The primary objective centers on developing a seamless, middleware-free interface between 3D Slicer and a physics engine. Per the specific operational requirements, this implementation will initiate using PyBullet to establish the kinematic foundation and rigid-body integration,

before transitioning to MuJoCo for advanced surgical tissue simulation. This necessitates establishing a shared memory architecture, synchronizing coordinate systems mathematically, and updating the 3D Slicer scene graph in real-time.

Comparative Engine Analysis: Transitioning from PyBullet to MuJoCo

The strategic roadmap mandates starting with PyBullet. PyBullet operates as an extremely lightweight, Python-accessible physics simulation library built on the Bullet Physics SDK, offering a shared memory C-API that achieves microsecond-level latency between the physics server and the Python client¹⁴. This makes PyBullet an ideal candidate for Phase 1, allowing developers to rapidly prototype the extraction of joint states and base poses to drive vtkMRMLTransformNode updates in 3D Slicer without incurring the overhead of complex soft-body solvers.

However, moving toward a full-fledged surgical simulator necessitates migrating to an engine capable of handling complex biomechanics. The following comparison illustrates the architectural imperative to transition to MuJoCo for Phase 2.

Feature Domain	PyBullet Engine	MuJoCo Engine	SOFA Framework (Industry Reference)
Primary Architectural Focus	Rigid body dynamics, rapid reinforcement learning environments, simple mass-spring systems ¹⁴ .	High-throughput biomechanics, contact-rich manipulation, continuous-time force evaluation ⁶ .	Medical simulation, pure Finite Element Method (FEM) soft tissue modeling ⁶ .
Soft Body Implementation	Limited to explicit mass-spring models; highly unstable for volumetric organs; supports basic cloth/ropes ¹⁹ .	Native flexcomp primitive for tetrahedral meshes; supports St. Venant-Kirchhoff (SVK) hyperelasticity ²¹ .	Extensive FEM libraries; supports complex topological changes including real-time surgical cutting and tearing ²² .
API and Latency	Python API via shared memory C-API; highly optimized for single-machine,	Native Python bindings providing zero-copy direct memory access to internal state	Python plugins available, but historically relies on a heavy C++ scene-graph

	low-latency execution ¹⁴ .	vectors and mass matrices ¹³ .	architecture ¹⁸ .
Constraint Solving Mechanics	Standard impulse-based discrete linear complementarity problem (LCP) solvers.	Soft constraint formulation via convex optimization; allows controlled violation of strict complementarity ¹⁷ .	Highly customizable collision pipelines utilizing Conjugate Gradient solvers; incurs extreme computational cost ¹¹ .

While SOFA has historically been the gold standard for surgical simulation due to its rigorous FEM capabilities²², integrating it into a modern, high-speed reinforcement learning and visualization pipeline is cumbersome due to its heavy C++ dependencies and lower execution speeds¹⁸. MuJoCo strikes the optimal balance, offering the flexcomp volumetric soft-tissue modeling required for surgical simulation while maintaining the execution speeds necessary for 1000 Hz haptic rendering¹¹.

Mathematical Synchronization of Coordinate Systems

A fundamental challenge in bridging 3D Slicer with external robotic simulators lies in resolving coordinate system discrepancies. Medical imaging platforms, including 3D Slicer and its underlying ITK infrastructure, inherently operate in the RAS (Right-Anterior-Superior) or LPS (Left-Posterior-Superior) coordinate spaces to align with patient anatomy⁹. Conversely, physics engines like PyBullet and MuJoCo, alongside standard URDF (Unified Robot Description Format) and MJCF (MuJoCo Modeling XML Format) robotic formats, utilize right-handed Cartesian spaces, often adopting a Z-up convention for world gravity⁹.

To accurately reflect the simulated robot's state within 3D Slicer, the transformation matrices extracted from the physics engine must undergo a rigorous basis change. The conversion from

an LPS transformation matrix T_{LPS} (commonly exported by physics engines) to an RAS transformation matrix T_{RAS} is achieved via conjugation with a diagonal reflection matrix M :

$$M = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The transformation is strictly computed as:

$$T_{RAS} = M \cdot T_{LPS} \cdot M$$

Within the 3D Slicer embedded Python environment, this is implemented programmatically by

extracting the pose arrays from PyBullet or MuJoCo, constructing a `vtkMatrix4x4`, applying the spatial conversion via NumPy matrix multiplication, and assigning it to the respective `vtkMRMLTransformNode` utilizing the `SetMatrixTransformToParent()` or `SetAndObserveMatrixTransformToParent()` APIs²⁸.

Threading Architecture and API Synchronization Strategy

To achieve low-latency updates without the ROS2 middleware, the bridge must execute entirely within the 3D Slicer Python interpreter, leveraging Slicer's embedded Python capabilities while strictly respecting the non-thread-safe nature of the VTK rendering pipeline and MRML scene³.

The optimal software architecture utilizes an asynchronous simulation loop paired with a synchronous rendering update:

1. **High-Frequency Physics Thread (Simulation):** The PyBullet or MuJoCo engine executes in a separate Python multiprocessing process or a dedicated background thread. This thread steps the physics simulation forward at a frequency of 500 Hz to 1000 Hz. High-frequency stepping is mathematically required to maintain numerical stability during contact-rich interactions and to prevent the constraint solver from diverging¹³.
2. **Zero-Copy Inter-Process Communication:** The kinematic state of the simulation (joint positions, link quaternions, and soft body vertex arrays) is written continuously to a pre-allocated shared memory buffer (`multiprocessing.shared_memory`) or accessed directly via MuJoCo's native zero-copy NumPy bindings¹³.
3. **Synchronous Slicer Update Loop (Rendering):** A `qt.QTimer` running explicitly on Slicer's main thread is configured to trigger at the monitor's vertical refresh rate (e.g., 60 Hz, corresponding to a ~16.6 ms interval)¹. The timer callback reads the latest state from the shared buffer, applies the LPS-to-RAS conversion, updates the `vtkMRMLTransformNode` hierarchies for rigid links, and calls `Modified()` to trigger a scene redraw without blocking the physics engine³¹.

This decoupled architecture ensures the physics engine is never bottlenecked by 3D Slicer's rendering pipeline, preserving the deterministic nature of the simulation while providing smooth, sub-20ms visual feedback to the surgical operator.

Objective 2: Engineering the Full-Fledged Surgical Simulator

Transitioning from a rigid kinematic bridge to a comprehensive surgical simulator necessitates the integration of highly deformable tissues, realistic surgical manipulator kinematics, surgical control constraints, and force-feedback haptics. This phase relies exclusively on MuJoCo's advanced solver dynamics.

Soft Body Deformations and Tissue Mechanics

Traditional robotic simulators model rigid bodies effectively, but surgical environments are dominated by highly deformable organs, vessels, and tissues. MuJoCo addresses this via its `flexcomp` primitive, which allows the instantiation of hyperelastic soft bodies derived directly

from tetrahedral meshes²¹.

The implementation requires a multi-stage pipeline:

1. **Patient-Specific Mesh Generation:** Volumetric meshes of organs are first segmented from CT or MRI data using 3D Slicer's "Segment Editor" module⁴. The resulting surface representations (STL/OBJ) are processed through a tetrahedral meshing library (e.g., fTetWild) to generate the volumetric meshes required by MuJoCo²¹.
2. **Hyperelastic Mathematical Modeling:** MuJoCo models these tetrahedral meshes utilizing a St. Venant-Kirchhoff (SVK) material law. The physics engine parameterizes this elasticity through standard biomechanical properties defined in the MJCF XML: Young's modulus (determining tissue stiffness) and Poisson's ratio (determining the tissue's incompressibility, typically near 0.45 for biological tissues)²⁰.
3. **Viscoelasticity and Damping:** Because MuJoCo lacks native non-linear viscoelasticity, the energy dissipation inherent in soft biological tissue is approximated using Rayleigh damping coefficients and internal friction mechanisms parameterized within the elasticity tags²⁰.

To render these dynamic deformations within 3D Slicer in real-time, the bridge must directly mutate the mesh geometry. Instead of reloading the entire model per frame, the Python bridge extracts the deformed vertex coordinate array (`flexvert_xpos`) from the MuJoCo state. It then utilizes the VTK API `vtkPolyData::GetPoints()->SetPoint()` to directly mutate the vertex positions in Slicer's memory, followed immediately by calling `Modified()` on the parent node to trigger a high-speed renderer update²⁰.

Kinematic Constraints: Enforcing the Remote Center of Motion (RCM)

Minimally invasive surgery (MIS) relies on laparoscopic instruments inserted through small incisions or trocars. To prevent catastrophic tissue trauma at the insertion site, surgical robots like the da Vinci Research Kit (dVRK) Patient Side Manipulator (PSM) are mechanically or software-constrained to operate under a Remote Center of Motion (RCM)³⁴. The RCM dictates a strictly fixed pivot point in Cartesian space around which the instrument shaft must rotate, restricting translation to the insertion axis alone³⁵.

Within the MuJoCo physics engine, enforcing the RCM requires specialized soft-constraint modeling:

1. **Geometric Constraints:** The RCM is modeled by defining a static site at the intended trocar coordinate. An equality constraint (specifically a connect or weld constraint) is established between the surgical tool shaft and the RCM site³⁷. By adjusting the constraint parameters, the engine permits only two rotational degrees of freedom (pitch and yaw) and a single prismatic degree of freedom (insertion depth) relative to the site, locking lateral translations³⁷.
2. **Inverse Kinematics Optimization:** When solving for joint trajectories from master manipulator inputs, the Inverse Kinematics (IK) solver must optimize for the desired end-effector pose while strictly residing within the null space of the RCM constraint Jacobian³⁹. MuJoCo's continuous-time convex optimization solver handles these equality constraints elegantly, ensuring that simulated patient injury from lateral trocar

movement is mathematically prevented¹⁷.

Surgical Virtual Fixtures and Admittance Control Formulation

To enhance surgical safety, virtual fixtures (VFs) are employed as software-generated constraints that physically restrict tool movement into "forbidden regions," such as vital vessels, nerve bundles, or tumor margins³. In 3D Slicer, these regions are intuitively defined via preoperative segmentations and registered into the MRML scene⁴.

Implementing VFs requires a sophisticated admittance control scheme within the simulation loop. Under free-space manipulation, the position of the master haptic device directly maps to the simulated surgical tool. However, when the tool approaches or breaches a defined margin, a reactive penalty force must be calculated and applied to the user's hand⁴¹.

The mathematical formulation relies on computing the minimum Euclidean distance between

the surgical tool tip position x_a and the boundary manifold of the forbidden region S ⁴². If

the tool penetrates the boundary, a reactive penalty force F_{pf} is generated based on a virtual mass-spring-damper model:

$$F_{pf} = \begin{cases} K_p(x_{boundary} - x_a) - B_p\dot{x}_a & \text{if } x_a \in \text{Forbidden Region} \\ 0 & \text{otherwise} \end{cases}$$

Where K_p represents the virtual stiffness coefficient of the boundary, and B_p represents the virtual damping coefficient⁴³. The admittance controller integrates this force, modifying the commanded velocity of the surgical robot to resist the user's motion and prevent further tissue penetration, thereby enforcing the active constraints defined by the 3D Slicer imaging data⁴⁴.

Haptic Rendering and Teleoperation Stability: The Time-Domain Passivity Approach

For a surgical simulator to be functionally viable for training and policy learning, haptic feedback must be transmitted to a master manipulator (e.g., a SensAble Phantom Omni, Force Dimension Sigma, or dVRK Master Tool Manipulator). The human tactile system is extraordinarily sensitive to delay and high-frequency vibrations; therefore, haptic control loops must operate at a strict minimum of 1000 Hz to maintain perceptual fidelity and system stability¹⁰.

A major theoretical challenge in interfacing a physical master device with a simulated virtual environment is the accumulation of spurious energy. Communication latency, discrete-time sampling, and the numerical integration of stiff contacts in the physics engine can inject active energy into the system¹¹. This excess energy violently violates the passivity criterion of the teleoperation system, leading to dangerous harmonic oscillations and hardware instability¹¹.

To guarantee stable bilateral teleoperation, the simulator must strictly implement the Time-Domain Passivity Approach (TDPA)¹¹. The TDPA functions as a dual-layer system monitoring the energy flow across the network ports connecting the master controller and the

simulated surgical slave.

The total energy $E_{obs}(k)$ observed at a discrete time step k is accumulated sequentially:

$$E_{obs}(k) = E_{obs}(k - 1) + (F(k) \cdot v(k)) \Delta t$$

If $E_{obs}(k) < 0$, the system is mathematically producing active energy, indicating a passivity violation. To counteract this and restore stability, a passivity controller (PC) injects a

variable damping element $\alpha(k)$ designed to dissipate the exact amount of excess energy generated during that time step¹¹:

$$\alpha(k) = \frac{-E_{obs}(k)}{v(k)^2 \Delta t}$$

The modified force $F_{cmd}(k)$ commanded to the physical haptic device is then adjusted:

$$F_{cmd}(k) = F(k) + \alpha(k)v(k)$$

By integrating the TDPA directly into the 1000 Hz physics loop within MuJoCo, the simulator guarantees unconditional stability regardless of the extreme stiffness of the simulated soft tissue, the high impedance of the Virtual Fixtures, or the computational latency of the 3D Slicer visualization updates¹⁰.

Exhaustive Task Breakdown and Implementation Roadmap

To transition these theoretical architectures and mathematical formulations into a functional, production-ready software suite, the project must be structured into three sequential phases. The following tables present an exhaustive, actionable breakdown of every engineering task required to implement the objectives defined by the project scope.

Phase 1: Establishing the Slicer-to-PyBullet Kinematic Bridge (Objective 1)

This phase establishes the middleware-free baseline, focusing on rigid body kinematics, shared memory initialization, and scene graph manipulation.

Task ID	Component	Engineering Action and API Utilization	Dependency
---------	-----------	--	------------

1.1	Environment Setup	Compile 3D Slicer from source to ensure full compatibility with deep Python integrations and C++ loadable modules, bypassing the security limitations of pre-compiled binaries ⁴ . Install the physics engines into Slicer's embedded Python environment using <code>slicer.util.pip_install('pybullet mujoco')</code> ³⁰ .	System Architecture
1.2	Robotic Asset Parsing	Establish a robust URDF/MJCF parser script within Slicer to dynamically read robotic descriptions. Translate these structural definitions into a programmatic hierarchy of <code>vtkMRMLModelNode</code> elements (for visual STL/OBJ links) and <code>vtkMRMLTransformNode</code> elements (for joint articulations) ⁴⁹ .	1.1
1.3	IPC Architecture	Instantiate a multi-threaded architecture within Python. Allocate	1.1

		Thread A (Simulation) for executing <code>pybullet.stepSimulation()</code> at 500 Hz. Design a zero-copy shared memory buffer (<code>multiprocessing.shared_memory</code>) to hold a flattened array of transformation matrices representing the current kinematic state¹³.	
1.4	Coordinate Conversion	Implement the LPS-to-RAS coordinate conversion matrix mathematics. Create a highly optimized, vectorized Python function that extracts the shared memory buffer, applies the T_{RAS} conjugation ($M \cdot T_{LPS} \cdot$), and formats the output into VTK-compatible <code>vtkMatrix4x4</code> structures²⁸.	1.2, 1.3
1.5	Slicer Sync Loop	Instantiate a Slicer <code>qt.QTimer</code> assigned to Slicer's main UI	1.4

		thread, set to an interval of ~16.6ms (60 Hz)¹. Inside the timer callback, iterate through all tracked vtkMRMLTransform Node elements and execute SetMatrixTransform ToParent() with the newly computed matrices, invoking scene updates efficiently without blocking the physics thread²⁸.	
--	--	--	--

Phase 2: MuJoCo Integration and Soft Body Dynamics (Objective 2a)

This phase transitions the physics backend to MuJoCo, enabling high-fidelity surgical tissue mechanics, hyperelasticity, and dynamic Slicer rendering.

Task ID	Component	Engineering Action and API Utilization	Dependency
2.1	Tetrahedral Meshing	Utilize 3D Slicer’s "Segment Editor" to isolate patient-specific organ geometries (e.g., liver, kidneys) from DICOM data⁴. Export the surface representations and process them through a tetrahedral meshing library (e.g., fTetWild) to generate	Medical Imaging Data

		volumetric meshes conforming to MuJoCo's flexcomp requirements ²¹ .	
2.2	MJCF Parameterization	Author the MJCF XML definition importing the tetrahedral mesh as a flexcomp entity. Parameterize the elasticity block, assigning appropriate biomechanical values for Young's modulus, Poisson's ratio, and material thickness²⁰. Configure MuJoCo to utilize the implicitfast integration scheme to maintain stability when simulating stiff, highly connected tetrahedral structures¹⁷.	2.1
2.3	Dynamic Mesh Updating	Map the soft body's flexvert_xpos array directly from the MuJoCo state to the corresponding vtkPolyData object in 3D Slicer²⁰. Modify the Phase 1 rendering callback: extract the updated vertices, apply the LPS-to-RAS	1.5, 2.2

		transform to the point cloud, and push the data into the VTK pipeline using surface->GetPoints()->SetData(). Conclude by calling Modified() on the MRML model node to force a redraw of the deformed tissue ³¹ .	
2.4	RCM Kinematics	Define surgical trocars as static collision bodies in the simulator. Implement the RCM constraint in MuJoCo by creating an equality node targeting the instrument shaft. Configure the weld constraint parameters to lock spatial translations while leaving the insertion axis and pitch/yaw rotations free, accurately simulating laparoscopic insertion limits ³⁴ .	2.2

Phase 3: Teleoperation, Admittance Control, and Haptics (Objective 2b)

The final phase closes the loop, integrating physical hardware, virtual safety boundaries, and passivity controllers to create a functional surgical simulator.

Task ID	Component	Engineering Action and API Utilization	Dependency
3.1	Hardware Interfacing	Establish a low-latency UDP socket or direct hardware API connection with a haptic master device (e.g., OpenHaptics for the Omni Bundle, or the dVRK research API) to stream operator joint and Cartesian commands directly into the Python physics thread ³ .	Phase 1
3.2	Virtual Fixture Admittance	Implement a high-speed collision detection algorithm within the MuJoCo simulation loop to monitor the distance between the surgical end-effector and Slicer-defined critical anatomical margins ⁴² . Develop the admittance controller script to calculate penalty forces linearly proportional to penetration depth and velocity. Output these resultant forces as	3.1

		impedance constraints onto the slave robot's joint torque commands ⁴⁴ .	
3.3	1000 Hz Haptic Loop	Isolate the haptic feedback calculation and the MuJoCo physics simulation step into a highly prioritized, synchronous thread targeting a strict 1ms execution time ¹¹ . Compute high-fidelity contact forces generated between the rigid surgical tool and the flexcomp soft tissue models by querying MuJoCo's continuous-time force evaluation arrays ¹⁷ .	3.2
3.4	TDPA Implementation	Implement the Time-Domain Passivity Approach (TDPA) energy observation algorithm. Continuously integrate the dot product of the commanded velocity and the returned interaction forces ¹¹ . Implement the passivity	3.3

		controller: if the energy state goes negative, compute the necessary dissipation variable α and dynamically alter the feedback force sent back to the physical master device¹¹.	
--	--	---	--

Conclusion

The development of a high-fidelity, image-guided robotic surgical simulator requires advancing beyond legacy middleware bridging paradigms to establish direct, low-latency communication between medical imaging platforms and modern physics engines. 3D Slicer provides an unparalleled environment for patient-specific imaging, segmentation, and spatial reasoning¹. Concurrently, the strategic integration of MuJoCo—following initial prototyping in PyBullet—provides the necessary computational throughput and advanced flexcomp tetrahedral modeling required for complex biomechanical interactions¹⁷.

Executing the proposed architectural roadmap guarantees a highly optimized, deterministic software suite. By isolating the 1000 Hz haptic and physics control loops from the 60 Hz 3D Slicer rendering thread, the system ensures physical accuracy without compromising visualization. Furthermore, by rigorously enforcing Remote Center of Motion (RCM) kinematics, Virtual Fixture admittance control, and the Time-Domain Passivity Approach (TDPA), researchers can construct a simulation environment capable of safely translating complex surgical robotic policies from the virtual domain into stable, real-world physical deployment.

Works cited

1. 2401.11715v1.pdf
2. sensors-22-05336-v2.pdf
3. 2509.19076v1.pdf
4. TREK: an integrated system architecture for intraoperative cone, <https://pmc.ncbi.nlm.nih.gov/articles/PMC9119410/>
5. Evaluating Simulation Platforms for Modular Mobile Robotic Systems, <https://www.mdpi.com/2075-1702/14/6/666>
6. Unfolding the Literature: A Review of Robotic Cloth Manipulation, <https://www.annualreviews.org/content/journals/10.1146/annurev-control-022723-033252?crawler=true&mimetype=application/pdf>
7. OpenPRC: A Unified Open-Source Framework for Physics-to-Task, <https://arxiv.org/html/2604.07423v1>
8. SlicerROS2: A research and development module for image-guided,

- <https://pmc.ncbi.nlm.nih.gov/articles/PMC12327204/>
9. Transforms - 3D Slicer documentation - Read the Docs,
https://slicer.readthedocs.io/en/latest/developer_guide/modules/transforms.html
 10. Toward Immersive Communications in 6G - arXiv,
<https://arxiv.org/html/2303.05283v1>
 11. Model-mediated Telemanipulation | Request PDF - ResearchGate,
https://www.researchgate.net/publication/220122877_Model_Mediated_Telemanipulation
 12. PyNoetic: A modular python framework for no-code development of,
<https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0327791>
 13. Neurosim: A Fast Simulator for Neuromorphic Robot Perception - arXiv,
<https://arxiv.org/html/2602.15018v1>
 14. Track Awesome Robotic Tooling Updates Daily,
<https://www.trackawesomelist.com/prototypes/awesome-robotic-tooling/>
 15. Webots Robot Simulator: Physics, Sensor Fidelity & ROS ... - SimTool,
<https://www.simtool.com/article/webots-open-source-robot-simulation-with-cross-platform-physics-and-sensor-fidelity/>
 16. Zero-shot Whole-Body Manipulation with a Large-Scale Soft Robotic,
<https://arxiv.org/html/2509.23556v1>
 17. Computation - MuJoCo Documentation,
<https://mujoco.readthedocs.io/en/stable/computation/index.html>
 18. Simulation of laparoscopic surgery using SOFA at interactive rates,
https://www.researchgate.net/figure/Simulation-of-laparoscopic-surgery-using-SOFA-at-interactive-rates-about-50Hz_fig1_6430523
 19. EthicalML/awesome-production-machine-learning: A curated list of,
<https://github.com/EthicalML/awesome-production-machine-learning>
 20. Flex Feature Parity Tracker: MuJoCo → MuJoCo Warp #1362 - GitHub,
https://github.com/google-deepmind/mujoco_warp/issues/1362
 21. Simulating the soft-tissue interface for a knee exoskeleton for,
<https://www.techrxiv.org/doi/pdf/10.36227/techrxiv.176972148.89375705/v1>
 22. Finite Element Analysis-Based Soft Robotic Modeling - ResearchGate,
https://www.researchgate.net/publication/369491201_Finite_Element_Analysis-Based_Soft_Robotic_Modeling_Simulating_a_Soft_Actuator_in_SOFA
 23. Modeling - MuJoCo Documentation,
<https://mujoco.readthedocs.io/en/latest/modeling.html>
 24. Applications of Reinforcement Learning for Autonomous Surgical,
<https://www.mdpi.com/2313-7673/11/8/577>
 25. A Real-Time Differentiable Physical Simulator for Soft Robotics,
https://www.researchgate.net/publication/335144650_ChainQueen_A_Real-Time_Differentiable_Physical_Simulator_for_Soft_Robotics
 26. RFUniverse: A Physics-based Action-centric Interactive Environment,
https://www.researchgate.net/publication/358292689_RFUniverse_A_Physics-based_Action-centric_Interactive_Environment_for_Everyday_Household_Tasks
 27. Documentation/Nightly/FAQ - Slicer Wiki,
<https://www.slicer.org/wiki/Documentation/Nightly/FAQ>

28. Transforms - 3D Slicer documentation - Read the Docs,
https://slicer.readthedocs.io/en/v4.11/developer_guide/modules/transforms.html
29. Transforms - 3D Slicer documentation - Read the Docs,
https://slicer.readthedocs.io/en/latest/developer_guide/script_repository/transforms.html
30. Script repository - 3D Slicer documentation - Read the Docs,
https://slicer.readthedocs.io/en/latest/developer_guide/script_repository.html
31. An Introduction To Programming For Medical Image Analysis With,
<https://www.scribd.com/document/88742006/An-Introduction-to-Programming-for-Medical-Image-Analysis-With-the-Visualization-Toolkit-Version-2-Updated-for-VTK-5-2>
32. (PDF) Generalized 3D nonlinear transformations for medical imaging,
https://www.researchgate.net/publication/10860232_Generalized_3D_nonlinear_transformations_for_medical_imaging_An_object-oriented_implementation_in_VTK
33. XML Reference - MuJoCo Documentation,
<https://mujoco.readthedocs.io/en/latest/XMLreference.html>
34. Human-in-the-Loop Embodied Intelligence With Interactive,
https://www.researchgate.net/publication/371432342_Human-in-the-loop_Embodied_Intelligence_with_Interactive_Simulation_Environment_for_Surgical_Robot_Learning
35. Orbit-Surgical: An Open-Simulation Framework for Learning ... - arXiv,
<https://arxiv.org/html/2404.16027v1>
36. Rudra Sharma | Robotics and AI Engineer, <https://roboticsage.info/>
37. Design and Performance Analysis of a Novel Ankle Rehabilitation,
<https://asmedigitalcollection.asme.org/mechanicaldesign/article/148/11/113308/1233052/Design-and-Performance-Analysis-of-a-Novel-Ankle>
38. Modeling - MuJoCo Documentation,
<https://mujoco.readthedocs.io/en/3.2.0/modeling.html>
39. ICCAS 2026 Program | Friday October 30, 2026,
https://ras.papercept.net/conferences/scripts/rtf/ICCAS2026_ContentListWeb_4.html
40. Enhancing surgical accuracy using virtual fixtures and motion,
https://robotics.ntua.gr/wp-content/uploads/sites/2/Enhancing_surgical_accuracy_using_virtual_fixtures_and_motion_compensation_in_robotic_beating_heart_surgery.pdf
41. Admittance Control for Adaptive Remote Center of Motion in Robotic,
<https://arxiv.org/html/2404.04416v1>
42. Vision Assisted Control for Manipulation Using Virtual Fixtures,
<https://www.cs.jhu.edu/CIRL/publications/pdf/bettini-final.pdf>
43. Research on safety strategy of surgical robot based on virtual fixtures,
<https://francispress.com/uploads/papers/aUYyH8TLVioTpL84RzrnREFzhRGCL1sxZXWTWfjP.pdf>
44. Impedance vs. Admittance Control in Surgery | PDF | Robotics - Scribd,
<https://www.scribd.com/document/950627709/DCM-1>
45. Dynamic virtual fixture on the Euclidean group for admittance-type,

- <https://pmc.ncbi.nlm.nih.gov/articles/PMC4041146/>
46. IROS 2025 Program | Thursday October 23, 2025,
https://ras.papercept.net/conferences/conferences/IROS25/program/IROS25_ContentListWeb_3.html
 47. Wearable robotics : systems and applications 9780128146606,
<https://dokumen.pub/wearable-robotics-systems-and-applications-9780128146606-0128146605-9780128146590.html>
 48. Adaptive 5G Low-Latency Communication for Tactile Internet Services,
https://www.researchgate.net/publication/327465133_Adaptive_5G_Low-Latency_Communication_for_Tactile_Internet_Services
 49. URDF importer for 3D Slicer - GitHub Gist,
<https://gist.github.com/lassoan/e174e853cd78ad93a1cd54b32debdac8>